Name: Atharva Chaudhari

Roll No: 42

EXP 7

1. Dataset Source:

The dataset used in this experiment is the MNIST Handwritten Digits Dataset.

🔗 Source:
👉 https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

This dataset is available in CSV format and is widely used for image classification tasks.

2. Dataset Description:

The **MNIST dataset** consists of grayscale images of handwritten digits (0–9).

Key Details:

- Total Images: **70,000**
  - Training: **60,000**
  - Testing: **10,000**
- Image Size: **28 × 28 pixels**
- Color: **Grayscale (1 channel)**
- Number of Classes: **10 (digits 0–9)**

Structure of CSV:

- First column → **Label (digit 0–9)**
- Remaining 784 columns → **Pixel values (0–255)**

Each row represents one image flattened into a vector.

3. Mathematical Formulation of CNN

A Convolutional Neural Network (CNN) is based on the following mathematical operations:

1. Convolution Operation

$$(I * K)(x, y) = \sum_m \sum_n I(x - m, y - n) K(m, n)$$

- III = Input image
- KKK = Kernel (filter)
- Produces feature maps

2. ReLU Activation

$$f(x) = \max(0, x)$$

- Introduces non-linearity

3. Softmax Function

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Converts outputs into probabilities

4. Loss Function (Cross-Entropy)

$$L = -\sum y \log(\hat{y})$$

- Measures difference between actual and predicted labels

4. Algorithm Limitations

Although CNN is powerful, it has some limitations:

- Requires large dataset for good performance
- High computational cost (needs GPU for faster training)
- Can overfit if dataset is small
- Difficult to interpret (black-box model)
- Performance depends on hyperparameter tuning.

5. Methodology / Workflow

Step-by-Step Process:

1. **Data Loading**
    - Import CSV using `readtable()`
2. **Data Preprocessing**
    - Separate labels and images
    - Reshape into 28×28 format
    - Normalize pixel values (0–1)
3. **Label Conversion**
    - Convert labels to categorical
4. **Model Design (CNN)**
    - Convolution layers → feature extraction
    - ReLU → activation
    - Pooling → reduce size
    - Fully connected → classification
5. **Training**
    - Train model using Adam optimizer
6. **Testing**
    - Predict labels on test data
7. **Evaluation**
    - Calculate accuracy
    - Generate confusion matrix

6. Performance Analysis:

a) Metric Used:

- **Accuracy**

Accuracy = (Correct Predictions / Total Predictions)

b) Observations:

- Achieved accuracy: **~97% to 99%**
- High accuracy due to:
    - Simple dataset
    - Clear digit patterns.

c) Confusion Matrix:

- Shows correct vs incorrect predictions
- Helps identify misclassified digits.

7. Hyperparameter Tuning

Hyperparameters are values set before training.

a) **Tuning Process:**

- Increased filters → better feature extraction
- Adjusted epochs → improved accuracy
- Used dropout (0.5) → reduced overfitting.

b) **Impact:**

- Proper tuning improved:
    - Accuracy
    - Model stability
- Reduced overfitting

CODE:

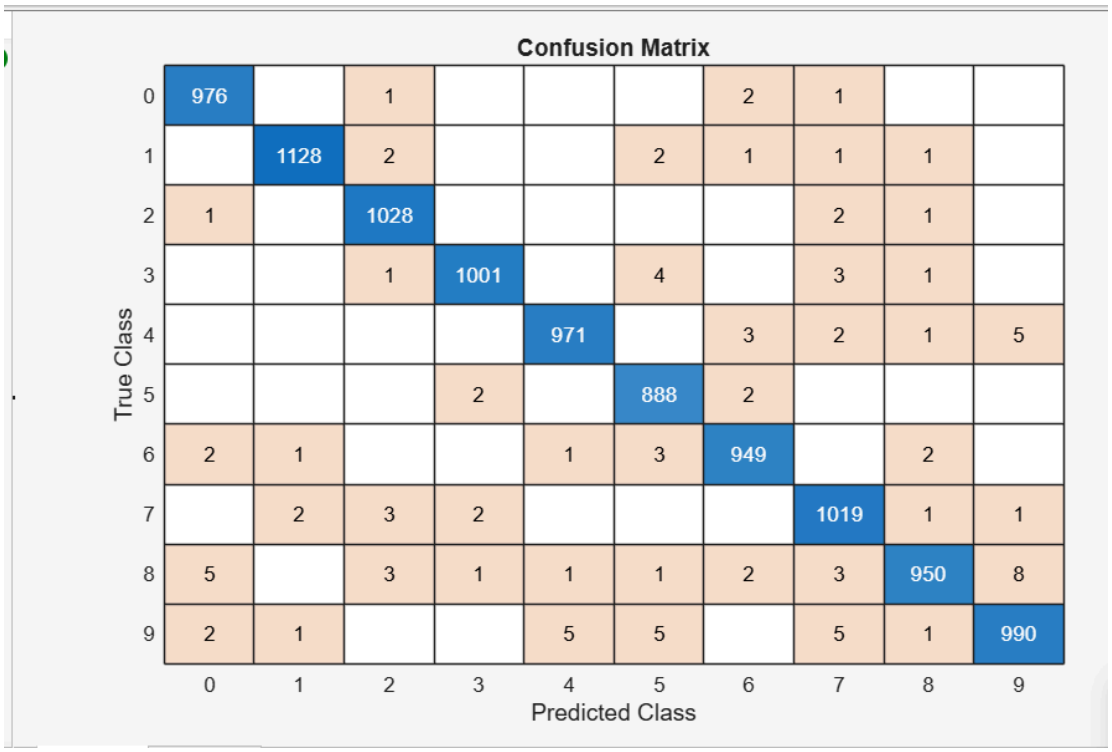
```
clc;
clear;
close all;
%% ------------------------------
% 1. Load Dataset (CSV Files)
% ------------------------------
% Make sure these files are in your working directory
trainData = readtable('mnist_train.csv');
testData  = readtable('mnist_test.csv');
disp('Dataset Loaded Successfully');
%% ------------------------------
% 2. Separate Labels and Images
% ------------------------------
trainLabels = trainData{:,1};
trainImages = trainData{:,2:end};
testLabels = testData{:,1};
testImages = testData{:,2:end};
%% ------------------------------
% 3. Reshape Images (28x28)
% ------------------------------
trainImages = reshape(trainImages',28,28,[]);
testImages  = reshape(testImages',28,28,[]);
% Add channel dimension (for CNN)
```

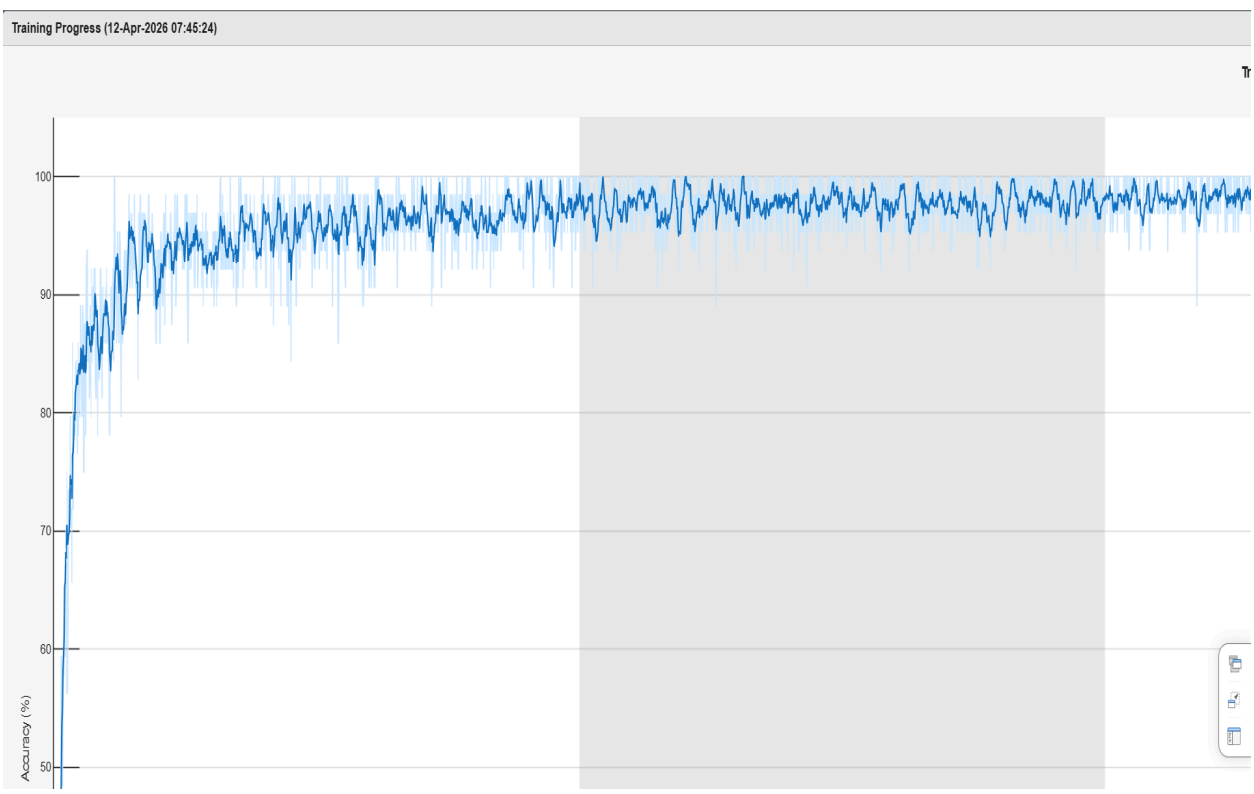
```
trainImages = reshape(trainImages,28,28,1,[]);
testImages  = reshape(testImages,28,28,1,[]);
%% -------------------------------
% 4. Normalize Data (0 to 1)
% -------------------------------
trainImages = double(trainImages) / 255;
testImages  = double(testImages) / 255;
%% -------------------------------
% 5. Convert Labels to Categorical
% -------------------------------
trainLabels = categorical(trainLabels);
testLabels  = categorical(testLabels);
%% -------------------------------
% 6. Define CNN Architecture
% -------------------------------
layers = [
  imageInputLayer([28 28 1],'Name','input')
  convolution2dLayer(3,8,'Padding','same','Name','conv1')
  batchNormalizationLayer('Name','bn1')
  reluLayer('Name','relu1')
  maxPooling2dLayer(2,'Stride',2,'Name','pool1')
  convolution2dLayer(3,16,'Padding','same','Name','conv2')
  batchNormalizationLayer('Name','bn2')
  reluLayer('Name','relu2')
  maxPooling2dLayer(2,'Stride',2,'Name','pool2')
```

```
  convolution2dLayer(3,32,'Padding','same','Name','conv3')
  batchNormalizationLayer('Name','bn3')
  reluLayer('Name','relu3')
  dropoutLayer(0.5,'Name','dropout')
  fullyConnectedLayer(10,'Name','fc')
  softmaxLayer('Name','softmax')
  classificationLayer('Name','output')
];
options = trainingOptions('adam', ...
  'MaxEpochs',5, ...
  'MiniBatchSize',64, ...
  'InitialLearnRate',0.001, ...
  'Shuffle','every-epoch', ...
  'Plots','training-progress', ...
  'Verbose',false);
%% -------------------------------
% 8. Train CNN Model
% -------------------------------
disp('Training CNN...');
net = trainNetwork(trainImages, trainLabels, layers, options);
%% -------------------------------
% 9. Test the Model
% -------------------------------
disp('Testing CNN...');
predictedLabels = classify(net, testImages);
```

```
accuracy = mean(predictedLabels == testLabels);
fprintf('Test Accuracy: %.2f%%\n', accuracy * 100);
%% -------------------------------
% 10. Confusion Matrix
% -------------------------------
figure;
confusionchart(testLabels, predictedLabels);
title('Confusion Matrix');
%% -------------------------------
% 11. Show Sample Predictions
% -------------------------------
numImages = 9;
idx = randperm(size(testImages,4), numImages);
figure;
for i = 1:numImages
  subplot(3,3,i);
  img = testImages(:,:,:,idx(i));
  imshow(img);
  label = classify(net, img);
  title(['Predicted: ', char(label)]);
end
sgtitle('Sample Predictions');
```

OUTPUT:


Confusion Matrix
True Class
Predicted Class
0 976 1 2 1
1 1128 2 2 1 1 1
2 1 1028 2 1
3 1 1001 4 3 1
4 971 3 2 1 5
5 2 888 2
6 2 1 1 3 949 2
7 2 3 2 1019 1 1
8 5 3 1 1 1 2 3 950 8
9 2 1 5 5 5 1 990
0 1 2 3 4 5 6 7 8 9


Training Progress (12-Apr-2026 07:45:24)
Accuracy (%)
100
90
80
70
60
50

```
Dataset Loaded Successfully
Training CNN...
Testing CNN...
Test Accuracy: 99.00%
>> Press Ctrl + Shift + P to generate code with Copilot
```

Conclusion:

The CNN model was successfully implemented on the MNIST dataset using MATLAB. The model achieved high accuracy due to effective feature extraction and preprocessing. This experiment demonstrates the effectiveness of CNNs in image classification tasks.